TECHRISE 3.0

Cybersecurity & DevSecOps · Abia Cohort 3.0 · 2026

WEEK 7 · MINI ASSIGNMENT

IDS/IPS · Snort 3.0 · Suricata · Wireshark · Network Security Monitoring

ANALYZED BY:

EZE CHUKWUEBUKA WILSON

TR3/STD/CYB/045

JULY, 2026

SECTION A — IDS/IPS FUNDAMENTALS

Question 1 (4 marks) — IDS vs IPS comparison table

Answer:

Feature	IDS	IPS
What it does when attack detected	Detects and generates an alert	Detects and blocks the attack
Network position	Mirror port (passive): Receives a copy of traffic	Inline: Sits directly in the traffic path
If the tool fails	Traffic keeps flowing with no detection	Network goes down
Risk level	Low — a false positive generates an alert with no actual attack	High — false positive blocks legitimate users

Question 2 (3 marks) — Four detection outcome types

Answer:

True Positive:

An alert fires and a real attack actually occurred. Example: Rule 3 (anonymous FTP) fired when I genuinely logged in with "USER anonymous" against the Metasploitable vsftpd service.

Example:

alert tcp any any -> $HOME_NET 21 (msg:"PAYLITENG Anonymous FTP Login Attempt"; flow:to_server,established; content:"USER anonymous"; nocase; sid:1000003; rev:1;)

False Positive:

An alert fires but there was no real attack. Example: the SQLi rule (UNION/SELECT content match) could fire on a legitimate DVWA user who happens to type those words in a comment field, even though no injection occurred.

Example:

alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS (msg:"PAYLITENG SQL Injection UNION SELECT"; flow:to_server,established; http_uri; content:"UNION", nocase; content:"SELECT", nocase, distance:0; sid:1000004; rev:1;)

False Negative:

A real attack occurs but no alert fires. Example: an SSH brute-force attempt that stays under the detection_filter threshold (e.g. 4 attempts in 60 seconds) would not trigger Rule 2, even though an attack was happening.

Example:
alert tcp any any -> $HOME_NET 22 (msg:"PAYLITENG SSH Brute Force Detected"; flow:to_server; detection_filter:track by_src, count 5, seconds 60; sid:1000002; rev:1;)

True Negative:

No alert fires and there was no attack. Example: normal browsing traffic to DVWA at the low security level, with no SQL keywords in any request, produces no alerts.

Question 3 (4 marks) — Rule analysis

Read this Snort 3.0 rule carefully and answer the questions below:

alert tcp any any -> $HOME_NET 22 (msg:"SSH Brute Force Detected"; flow:to_server; detection_filter:track by_src, count 5, seconds 60; sid:1000002; rev:1;)

a. What does 'flow:to_server' do?

Answer:

It restricts the rule to packets travelling in the client-to-server direction of the SSH service.

b. What does the detection_filter do — when exactly does this rule fire?

Answer:

It counts how many times the base rule condition (any TCP packet to port 22) matches per source IP (track by_src). The alert only fires once that source has matched 5 times within a 60-second window.

c. Why use detection_filter instead of firing on every SSH attempt?

Answer:

Firing on every attempt would flood the analyst with alerts for normal and occasional login attempts. It also reduces false positives and alert fatigue.

d. What would you change in this rule to make it fire after 3 attempts in 30 seconds instead?

Answer:

I would change the count and seconds to be 3 and 30 respectively; making the rule to become:

alert tcp any any -> $HOME_NET 22 (msg:"SSH Brute Force Detected"; flow:to_server; detection_filter:track by_src, count 3, seconds 30; sid:1000002; rev:1;)

Question 4 (4 marks)

What are the THREE types of IDS/IPS based on detection location? For each one name the type, what it monitors, and which tool from your Week 7 labs represents it:

Answer:

1. **NIDS (Network Intrusion Detection System)**
   Monitors and detects traffic across an entire network segment via a mirror port.
   Tool: Snort and Suricata.
2. **HIDS (Host Intrusion Detection System)**
   Monitors a single host including its logs, file integrity, and running processes
   Tool: Wazuh.
3. **NIPS (Network Intrusion Prevention System)**
   Sits inline with the traffic and actively blocks malicious packets rather than just alerting.
   Tool: Snort/Suricata run in inline/IPS mode instead of passive IDS mode.

SECTION B — Snort 3.0 and Suricata

Question 5 (6 marks):

Write a Snort 3.0 rule for each scenario below. Each rule must be syntactically correct and on a single line:

a. Detect ANY ICMP traffic (sid:2000001)

Answer:

alert icmp any any -> any any (msg:"TECHRISE ICMP Detected"; sid:2000001; rev:1;)

b. Detect anonymous FTP login attempts on port 21 (sid:2000002)

Answer:

alert tcp any any -> $HOME_NET 21 (msg:"PAYLITENG Anonymous FTP Login Attempt"; flow:to_server,established; content:"USER anonymous"; nocase; sid:2000002; rev:1;)

c. Detect SQL injection (UNION + SELECT) in HTTP URI using sticky buffer syntax (sid:2000003)

Answer:

alert tcp any any -> $HTTP_SERVERS $HTTP_PORTS (msg:"PAYLITENG SQL Injection UNION SELECT"; flow:to_server,established; http_uri; content:"UNION", nocase; content:"SELECT", nocase, distance:0; sid:2000003; rev:1;)

Question 6 (4 marks)

The table below shows a rule written for Snort 3.0. Rewrite it correctly for Suricata and explain what changed and why:

Answer:

Snort 3.0 version:

alert tcp any any -> any any (msg:"SQL Injection"; flow:to_server,established; http_uri; content:"UNION", nocase; content:"SELECT", nocase, distance:0; sid:3000004; rev:1;)

Suricata version:

Alert http any any -> any any (msg:”SQL Injection”; flow:to_server,established;http.uri; content:”UNION”; nocase; content:”SELECT”; nocase; distance:0;sid:3000004; rev:1;)

Explain ALL differences between the two versions:

1. Protocol keyword: Snort uses the generic transport protocol "tcp" plus the sticky buffer http_uri to inspect the HTTP layer; Suricata instead names the application-layer protocol directly as "http" in the header, since Suricata's protocol detection engine tracks HTTP natively regardless of port.

2. Sticky buffer naming: Snort 3's buffer is written http_uri (underscore); Suricata's equivalent is http.uri (dot notation), reflecting Suricata's namespaced keyword style for its many http.* buffers (http.method, http.header, etc.).

3. Option chaining style: Snort 3 commonly chains modifiers onto a content match using commas within the same option (content:"UNION", nocase), while Suricata generally expects each modifier as its own semicolon-separated option (content:"UNION"; nocase;) applying to the buffer currently in scope.

These differences exist because the two engines evolved independently — Snort 3 modernised Snort 2's tcp+sticky-buffer model, while Suricata was built from the start around explicit application-layer protocol parsing.

Question 7 (4 marks):

Answer these questions about Snort 3.0 and Suricata configuration:

a. Snort 3.0 configuration file name and how it differs from Snort 2.x

Answer:

The file is snort.lua. Unlike Snort 2.x's snort.conf (a flat, custom text format), snort.lua is written in the Lua scripting language, giving it structured tables, native variables, and the ability to include/reuse configuration blocks programmatically.

b. What is the correct Snort 3.0 command to validate your configuration and rules?

Answer:

snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules --warn-all -T

c. What does suricata-update do and why does it matter for a production SOC?

Answer:

suricata-update downloads and installs updated rulesets (such as the Emerging Threats Open ruleset) into Suricata's rules directory and rebuilds the active rule file.

It matters for a production SOC because new attack techniques, malware signatures, and CVEs appear constantly; without regularly updated rules, Suricata's detection coverage delays and misses current threats.

d. What is EVE JSON and why is it more useful than alert_fast.txt for Splunk?

Answer:

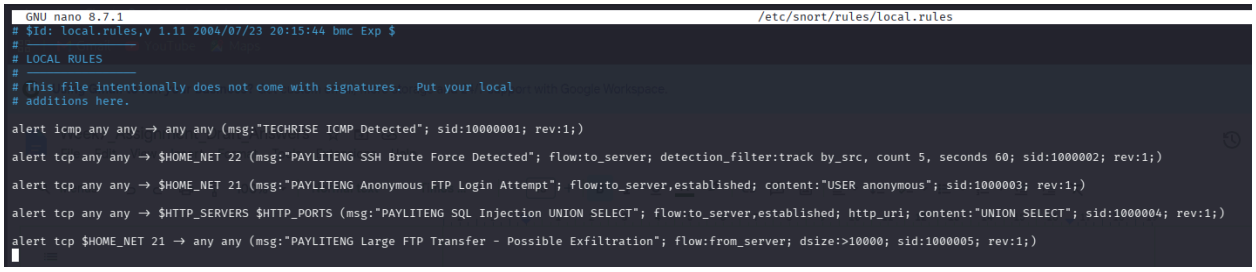
EVE JSON is Suricata's structured event log format, where every alert, HTTP request, DNS query, flow, etc. is logged as a JSON object with named fields. This is more useful for Splunk than Snort's flat alert_fast.txt because Splunk can automatically parse and index each named field, enabling searches, dashboards, and correlation directly on fields (e.g. src_ip, http.url) instead of relying on fragile text/regex parsing of unstructured lines.

Question 8 (6 marks) — Practical Evidence

Practical Evidence — paste your actual output for each of the following:

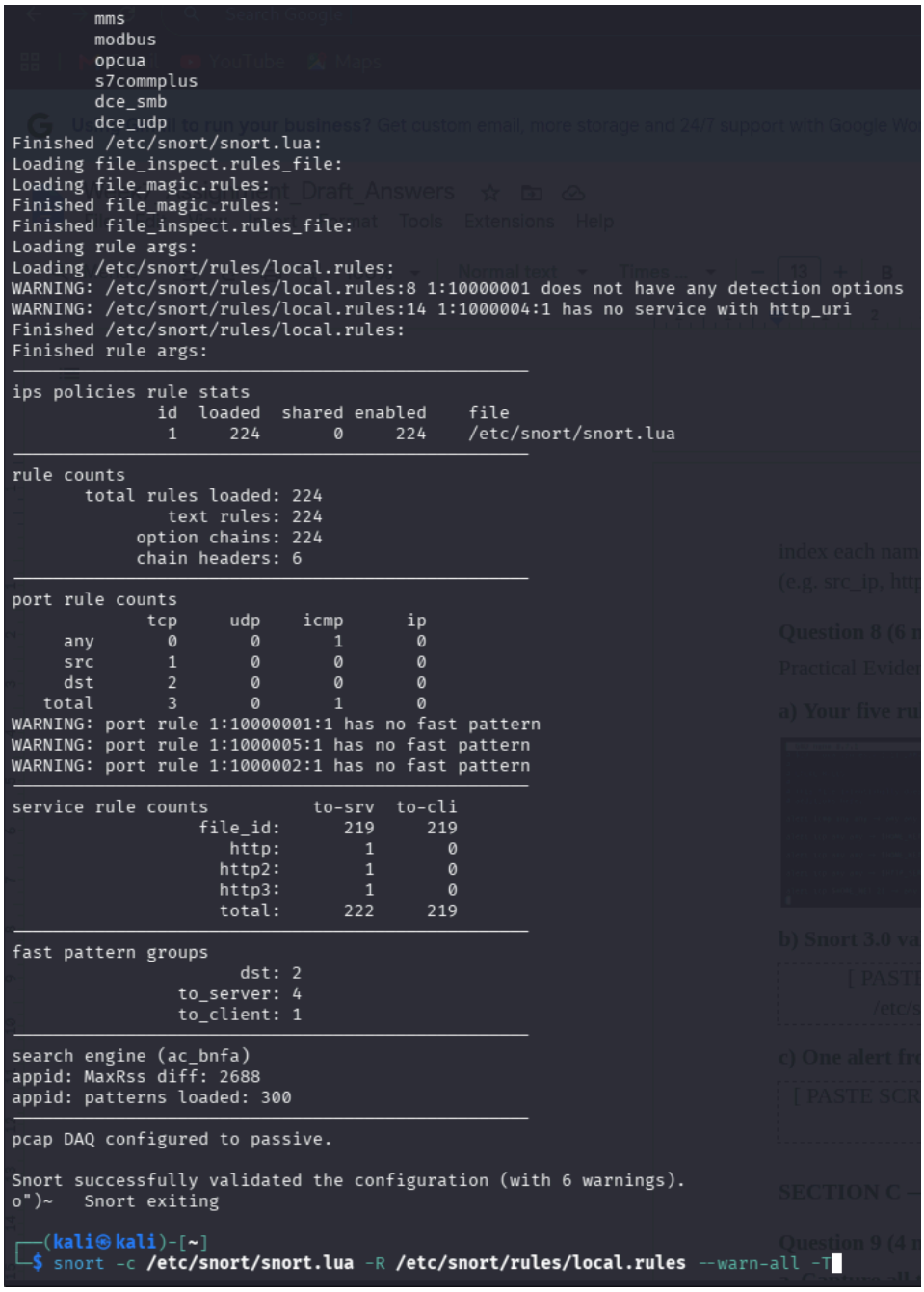
a) Your five rules from /etc/snort/rules/local.rules (2 marks)

Answer:



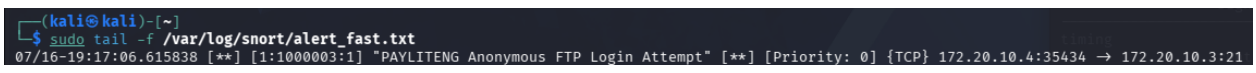
b) Snort 3.0 validation output showing rules loaded (2 marks)

Answer:



c) One alert from /var/log/snort/alert_fast.txt showing a rule firing (2 marks)

Answer:



SECTION C: NETWORK SECURITY MONITORING — tcpdump and PCAP

Write the exact tcpdump command for each task:

Question 9 (4 marks) — tcpdump commands

a. Capture all traffic on eth0 and save to evidence.pcap

Answer:

sudo tcpdump -i wlan0 -w evidence.pcap

b. Read from a saved PCAP and show only traffic on port 21

Answer:

sudo tcpdump -r evidence.pcap port 21

c. Capture traffic from a specific IP and display contents in ASCII

Answer:

sudo tcpdump -i wlan0 host 192.168.1.50 -A

d. Show only SSH traffic (port 22) in verbose mode

Answer:

sudo tcpdump -i wlan0 -v port 22

Question 10 (3 marks):

Explain what 'order of volatility' means in digital forensics and list FOUR types of evidence in order from most volatile to least volatile:

Answer:

Order of volatility means:
Collecting digital evidence in the order of what is most likely to disappear or change first, so that the most perishable/short-lived evidence is captured before it is lost, before moving on to more permanent evidence.

Most volatile (collect first): RAM / memory contents

Second: Network connections (active connections, routing tables, ARP cache)

Third: Running processes

Least volatile (can wait): Disk / storage (files, logs, archived data)

Question 11 (3 marks):
You captured a PCAP file during a suspected FTP data theft. Write the command to run Suricata against the PCAP file and explain what advantage this has over only having log files:

Answer:

Command:

suricata -c /etc/suricata/suricata.yaml -r capture.pcap -l /var/log/suricata/

Advantage over log files alone:
Replaying the PCAP lets you re-run detection (including updated or new rules) against the original raw traffic at any time, reproducing and verifying findings rather than trusting only a summarised log line. The PCAP is also portable, shareable evidence that other analysts or a court can independently examine, without needing live access to the original network.

SECTION D: Wireshark Basics - Network Packet Analysis

Wireshark shows packets organised in columns. Match each column to what it displays: Column Name Answer (A-E) Options No. _____ A. The protocol used — HTTP, TCP, DNS, FTP Time _____ B. A brief description of what the packet contains Source _____ C. The sequential packet number Protocol _____ D. The IP address the packet came from Info _____ E. How many seconds since capture started

Question 12 (3 marks) — Column matching

Answer:

Column Name	Answer
No.	C — The sequential packet number
Time	E — How many seconds since capture started
Source	D — The IP address the packet came from
Protocol	A — The protocol used — HTTP, TCP, DNS, FTP
Info	B — A brief description of what the packet contains

Question 13 (4 marks) — Wireshark display filters

a. Show only HTTP traffic

Answer:

http

b. Show only traffic from IP 185.220.101.47

Answer:

ip.addr == 185.220.101.47

c. Show only FTP traffic on port 21

Answer:

ftp

d. Show only TCP traffic going TO port 22 (SSH)

Answer:

tcp.dstport == 22

e. Show only DNS traffic

Answer:

dns

Question 14 (4 marks)

A forensic investigator opens a PCAP file in Wireshark and sees the following packets. Answer the questions below:

No.	Time	Source	Destination	Protocol	Info
1	03:47:15	185.220.101.47	192.168.43.100	TCP	22 → 54321 [SYN]
2	03:47:15	192.168.43.100	185.220.101.47	TCP	54321 → 22 [RST]
3	03:47:16	185.220.101.47	192.168.43.100	TCP	22 → 54322 [SYN]
4	03:47:16	192.168.43.100	185.220.101.47	TCP	54322 → 22 [RST]
5	03:51:44	185.220.101.47	192.168.43.100	FTP	Request: USER anonymous
6	03:51:45	192.168.43.100	185.220.101.47	FTP	Response: 230 Login successful
7	03:51:48	185.220.101.47	192.168.43.100	FTP	Request: RETR /backups/customer_backup.sql.gz
8	03:51:52	192.168.43.100	185.220.101.47	FTP-DATA	FTP Data Transfer 47823912 bytes

a. What is happening in packets 1 and 2? What does [RST] mean?

Answer:

Packet 1 is a SYN sent from 185.220.101.47 to port 22 (SSH) on the target, attempting to open a TCP connection. Packet 2 is an immediate RST sent back from the target, meaning the connection was forcibly refused/reset rather than accepted; the target is not completing the handshake for this attempt; RST means RESET.

b. Wireshark display filter to see only the FTP traffic in this capture

Answer:

Ftp

c. Packet 8 shows a 47,823,912 byte transfer — convert to MB and explain

Answer:

47,823,912 bytes ÷ 1,048,576 = 4.561 MB.

This represents the size of the customer_backup.sql.gz file being exfiltrated out of the network over the FTP-DATA channel after the RETR command in packet 7; meaning a database backup theft.

d. Which Snort 3.0 rule from the Week 7 labs would have detected packet 5? Rule message:

Answer:

Rule 3, "Anonymous FTP Login Attempt" — it matches on the content "USER anonymous" in FTP traffic to port 21, which is exactly what packet 5 shows.

Question 15 (4 marks) — Wireshark short questions

a. Difference between Wireshark and tcpdump; when to use each

Answer:

Wireshark is a graphical tool that displays captured packets with a full, layer-by-layer protocol breakdown and supports rich interactive filtering; tcpdump is a lightweight command-line capture tool with no GUI. We use tcpdump for quick or remote/headless captures and scripting; use Wireshark when you need to visually inspect, filter, and deeply analyse traffic, including PCAPs tcpdump already captured.

b. What does 'Follow TCP Stream' show and why is it useful in forensics?

Answer:

It reassembles every packet belonging to one TCP conversation into a single readable transcript of both sides of the exchange (e.g. the full FTP login, commands, and responses). This is useful in forensics because it lets an investigator read the entire session as it happened, rather than piecing it together packet by packet.

c. How do you open a PCAP captured with tcpdump -i eth0 -w capture.pcap in Wireshark?

Answer:

Open Wireshark, then use File → Open and click capture.pcap

or

run <wireshark capture.pcap> from the terminal.

d. Wireshark filter to find only HTTP requests containing 'UNION' in the URL:

Answer:

http.request.uri contains "UNION"